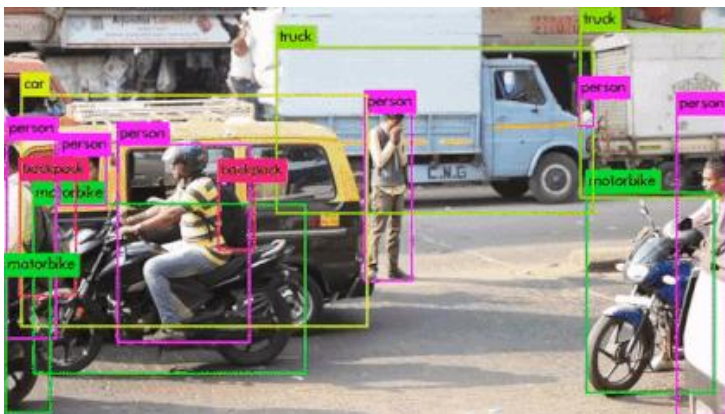


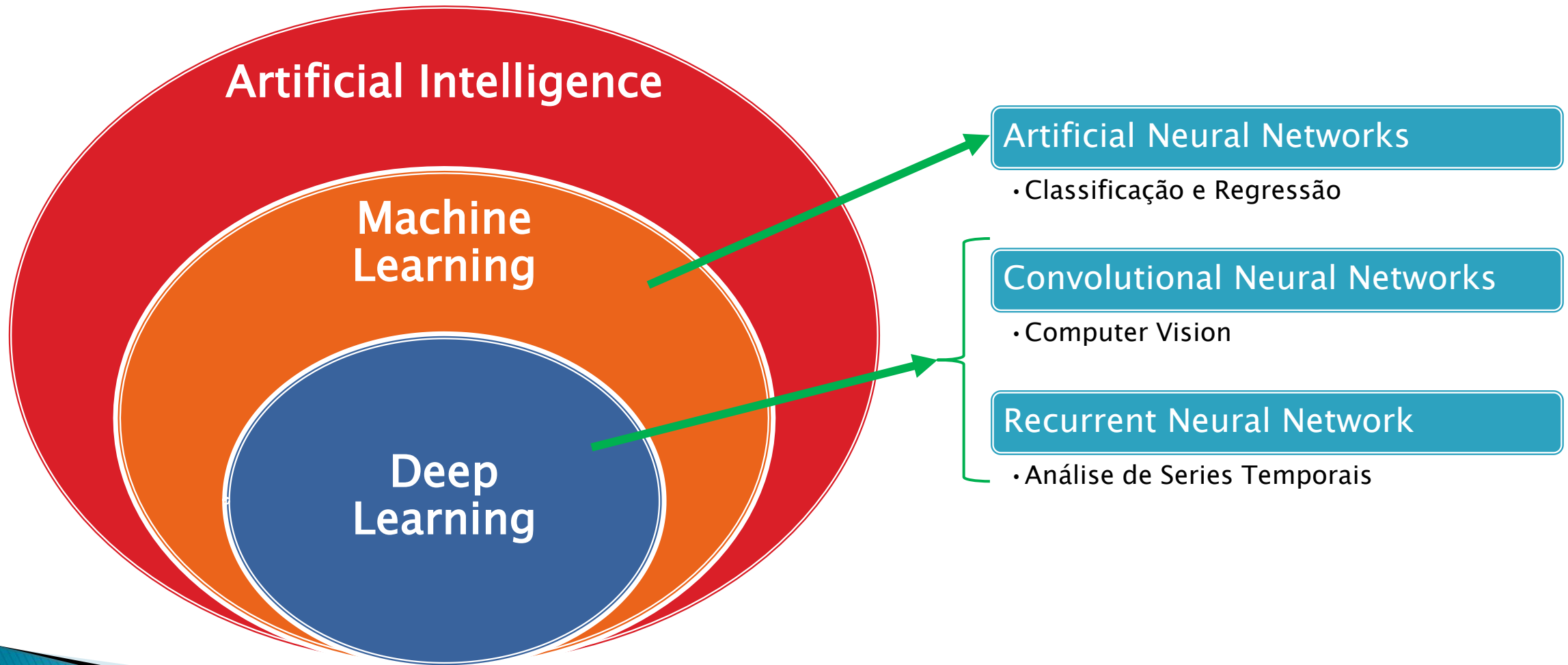


Reconhecimento de Padrões Avançado

Redes Neuronais Convolucionais

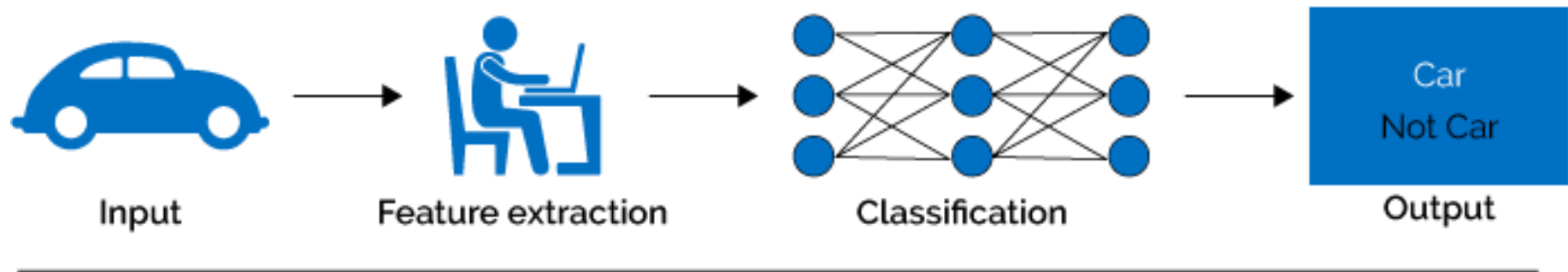


Data Science – enquadramento

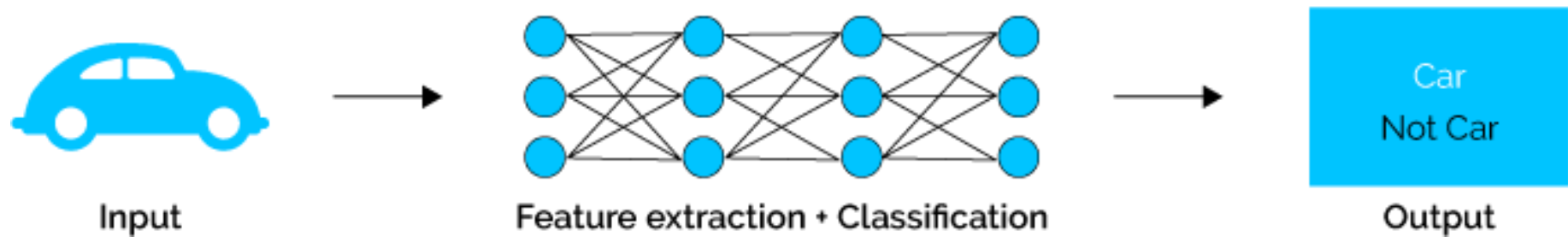


Processamento Clássico vs Deep Learning

Machine Learning



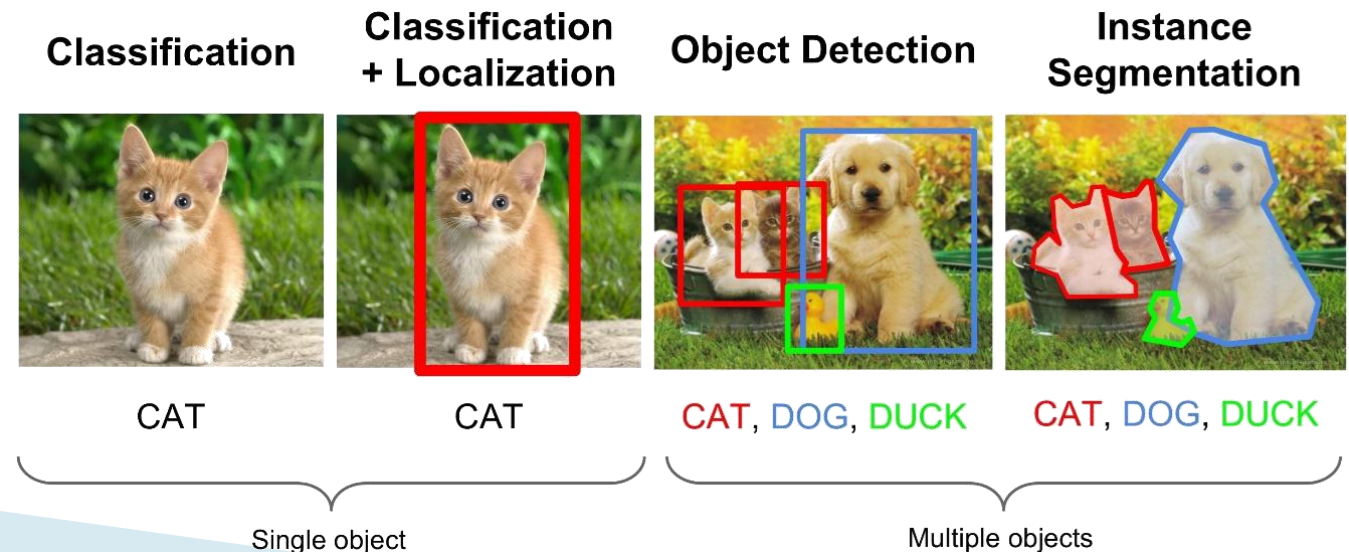
Deep Learning



Deep Learning Detectors

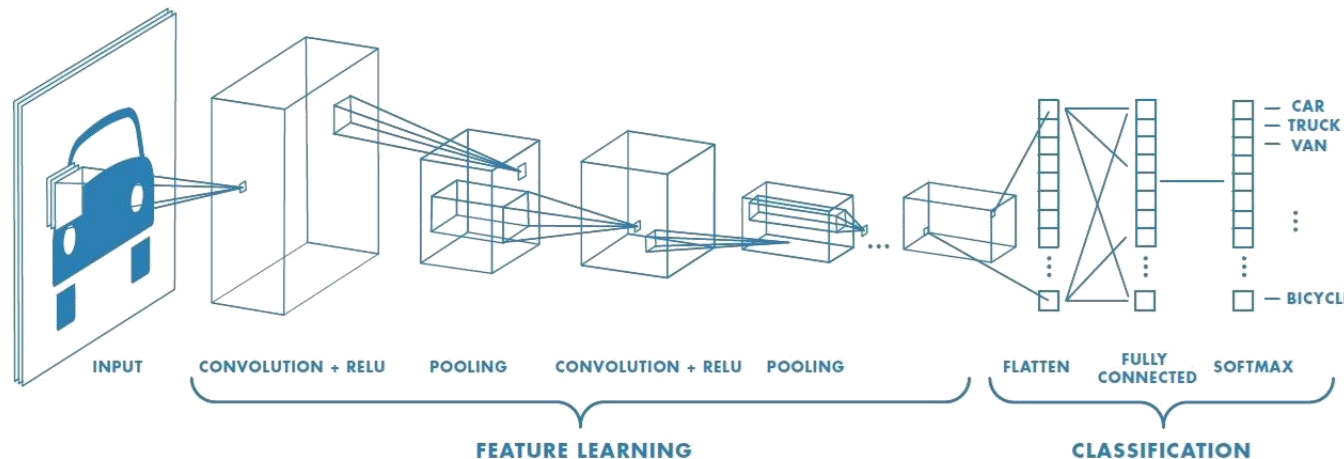
► Nível de detalhe dos detetores:

- **Image Classification** – Estimar o tipo ou classe de um objeto numa imagem.
- **Object Localization** – Localizar a presença de objetos numa imagem e indica a sua localização com uma caixa de delimitação.
- Object Detection** – Localiza a presença de objetos com uma caixa de delimitação e tipos ou classes dos objetos localizados numa imagem.
- **Object Instance Segmentation** – objetos reconhecidos são apresentados realçando os pixels específicos do objeto em vez de uma caixa de delimitação.



Redes Neurais Convolucionais

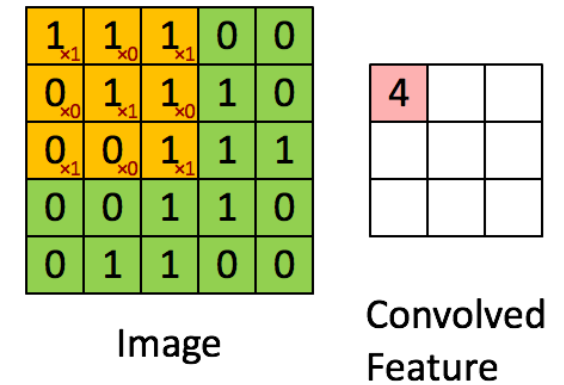
- ▶ Estas redes são atualmente os **detetores de última geração**
- ▶ Com treino suficiente podem aprender a filtrar e a aplicar muitas outras operações de imagem
 - Conjuntos de imagens de treino populares: PASCAL VOC, COCO, MNIST, ImageNet
- ▶ As camadas principais são: **Convolução, Pooling e Classificação** (fully connected layers – camadas totalmente ligadas)



Comportamento das camadas

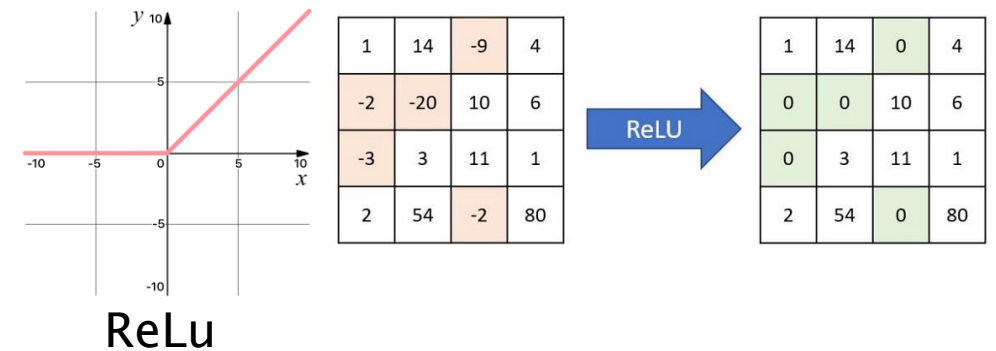
▶ Camadas de convolução

- Extrair vários mapas de características
- Convolucionar com diferentes núcleos



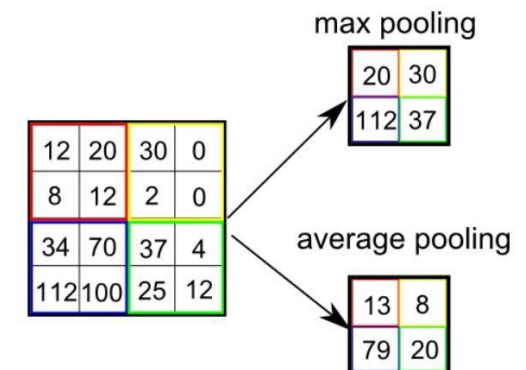
▶ Camada ReLu

- É uma função de ativação aplicada depois da convulsão para introduzir uma não linearidade
- Previne os *vanish gradients*



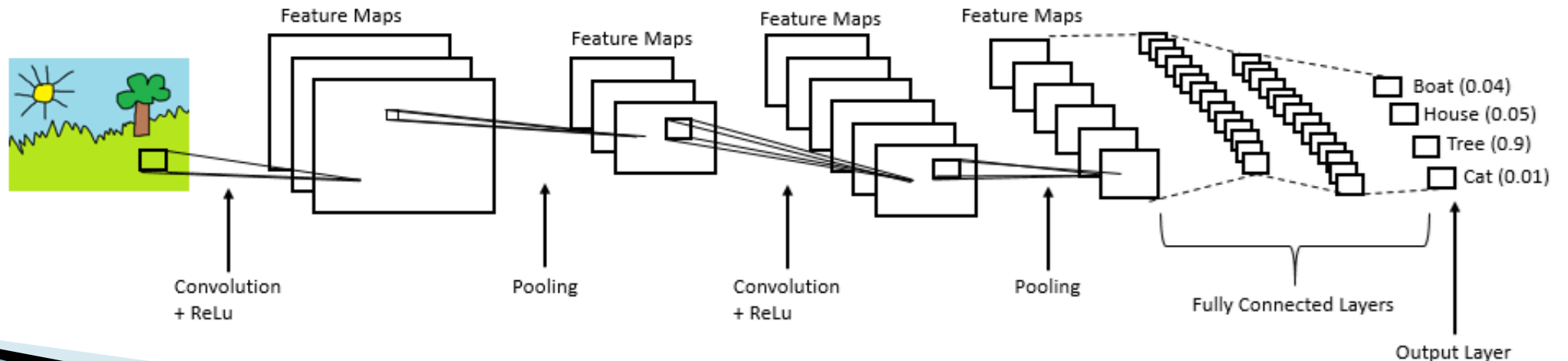
▶ Camadas de Pooling

- Reduzir o tamanho espacial dos mapas de características



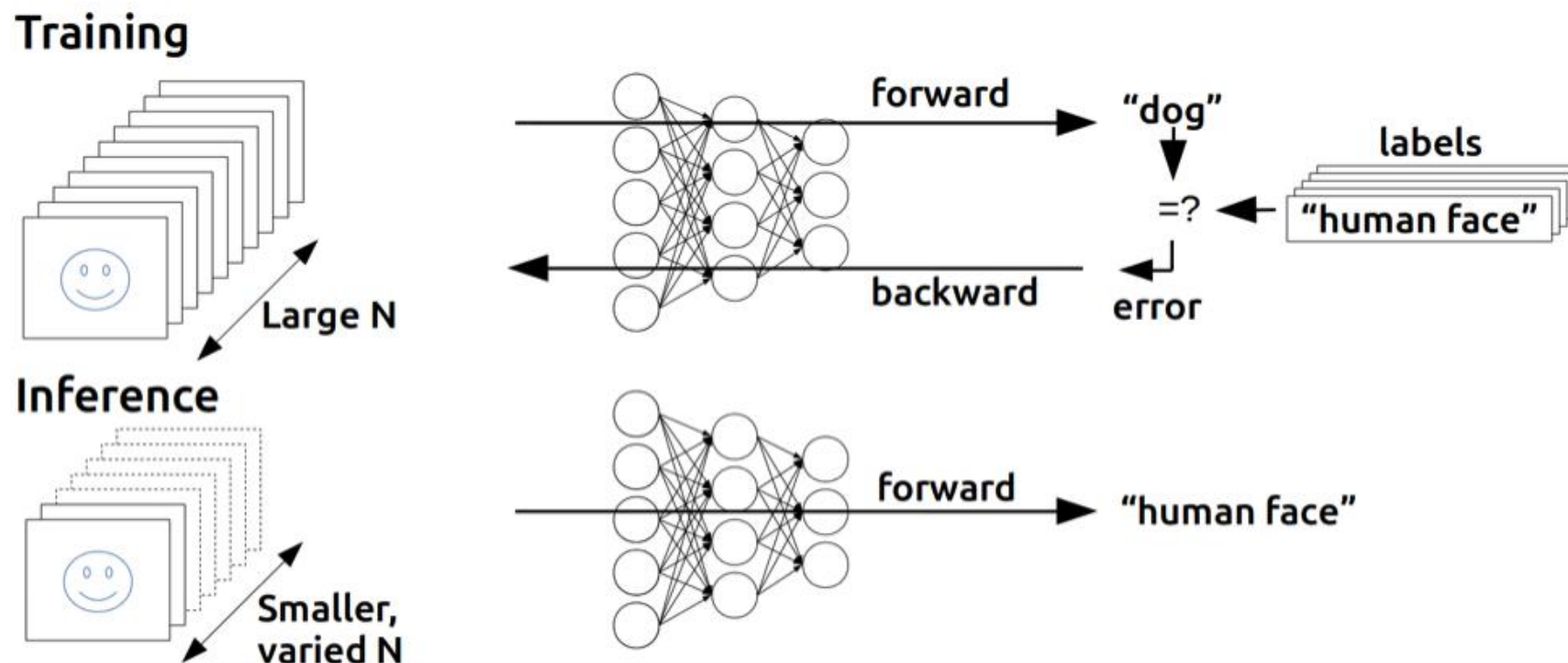
Classificação

- ▶ Para preparar os mapas para as camadas totalmente conectadas (*fully connected layers*) é necessário linearizar/"achatar" as matrizes
- ▶ Em seguida, é aplicado um classificador Softmax para realizar a classificação final
- ▶ A saída de um softmax é um conjunto de probabilidades para cada classe



Treino vs Inferência

- ▶ A fase de **treino** é executada uma vez para gerar o modelo
- ▶ A **inferência** é a aplicação do modelo a novas imagens



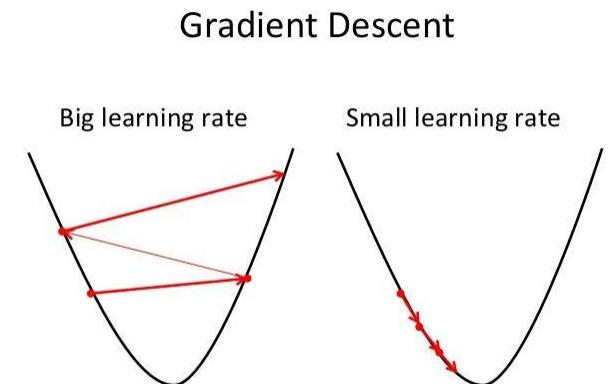
© Nvidia – <https://developer.nvidia.com/blog/inference-next-step-gpu-accelerated-deep-learning/>

Treino

- ▶ O treino consiste em ajustar os pesos da rede neuronal com o objetivo de reduzir o erro:

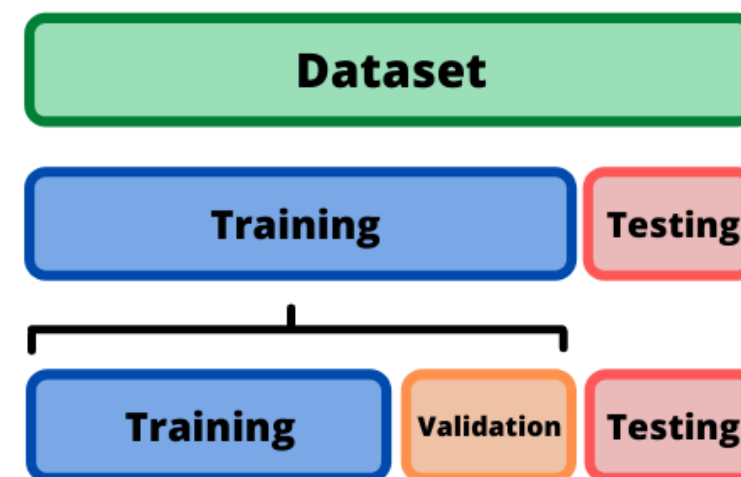
$$Erro = \sum \text{classificação esperada} - \text{classificação obtida}$$

- ▶ Os **Hiper parâmetros** de uma rede são configurações que podem ser ajustadas antes de iniciar o treino e podem ser:
 - Estrutura da Rede:
 - N° de **camadas** da rede e n° de **nós** por camada
 - Funções de **ativação**
 - Controlo do Treino:
 - **Fator de aprendizagem** (*learning rate*)
 - **Número de épocas** (epochs) – quantas vezes o todo o conjunto de treino é apresentado à rede
 - **Tamanho do lote** (*Batch size*) – n° de exemplos de treino usados para estimar o erro, antes de modificar os pesos
 - entre outras...



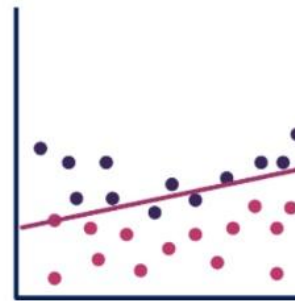
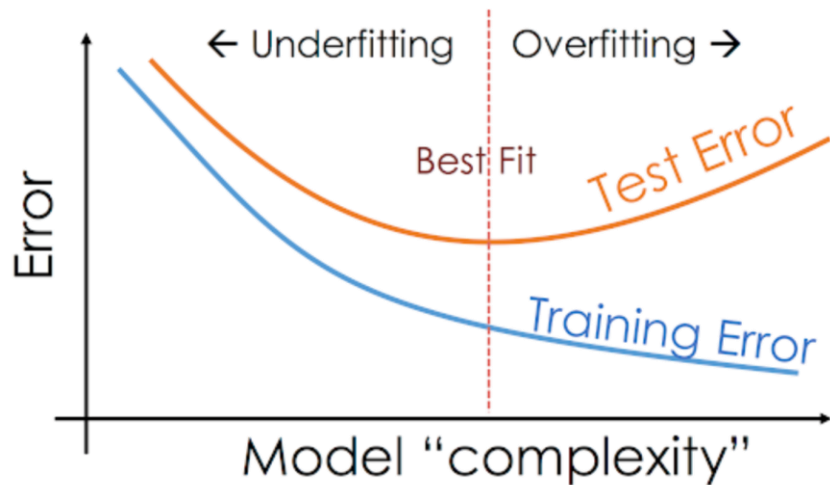
Avaliação de um classificador

- ▶ Para uma correta e imparcial avaliação de um classificador são definidos 3 conjuntos de imagens:
 - **Conjunto de Treino** – imagens usadas para ajustar os pesos do classificador
 - **Conjunto de Validação** – imagens usadas durante o treino para aferir a qualidade do classificador e fazer ajustes aos Hiper parâmetros
 - **Conjunto de Teste** – imagens usadas depois do classificador estar totalmente treinado para aferir a sua performance com imagens nunca antes vistas

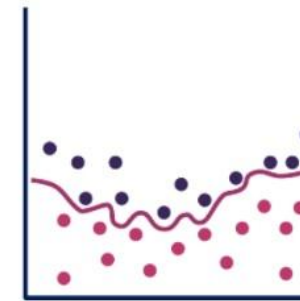


Overfitting

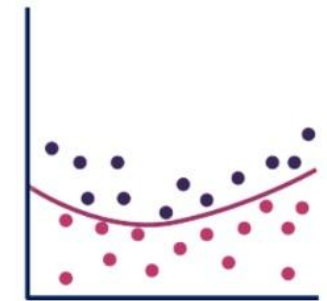
- ▶ *Overfitting* ocorre quando o modelo não consegue generalizar uma solução e fica demasiado ajustado às imagens de treino
- ▶ O resultado é que a performance para imagens novas (desconhecidas) irá ser baixa comparativamente às imagens de treino
- ▶ O limiar é quando o erro de teste começa a crescer, enquanto o erro de treino desce



Underfitting



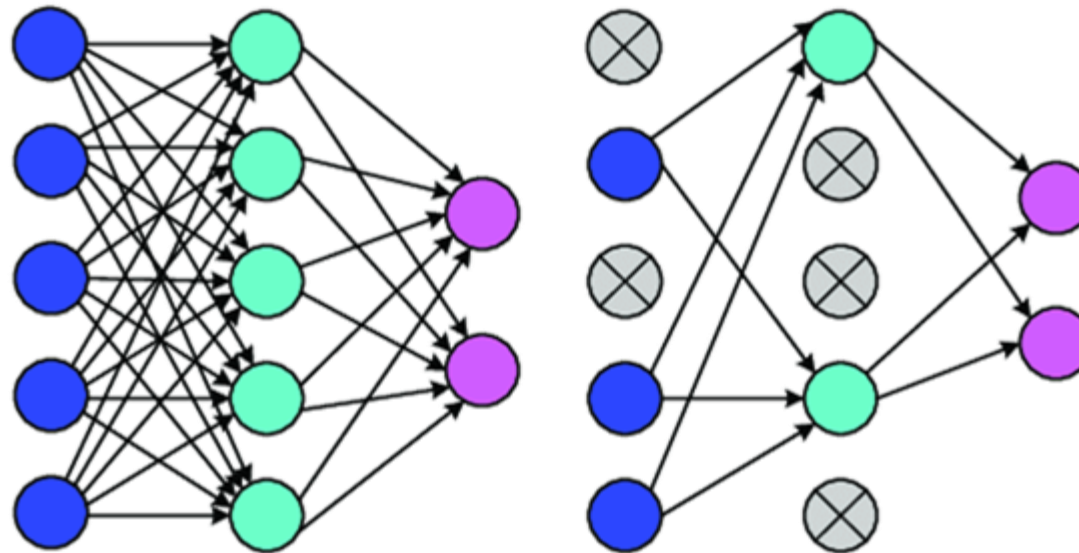
Overfitting



Balanced

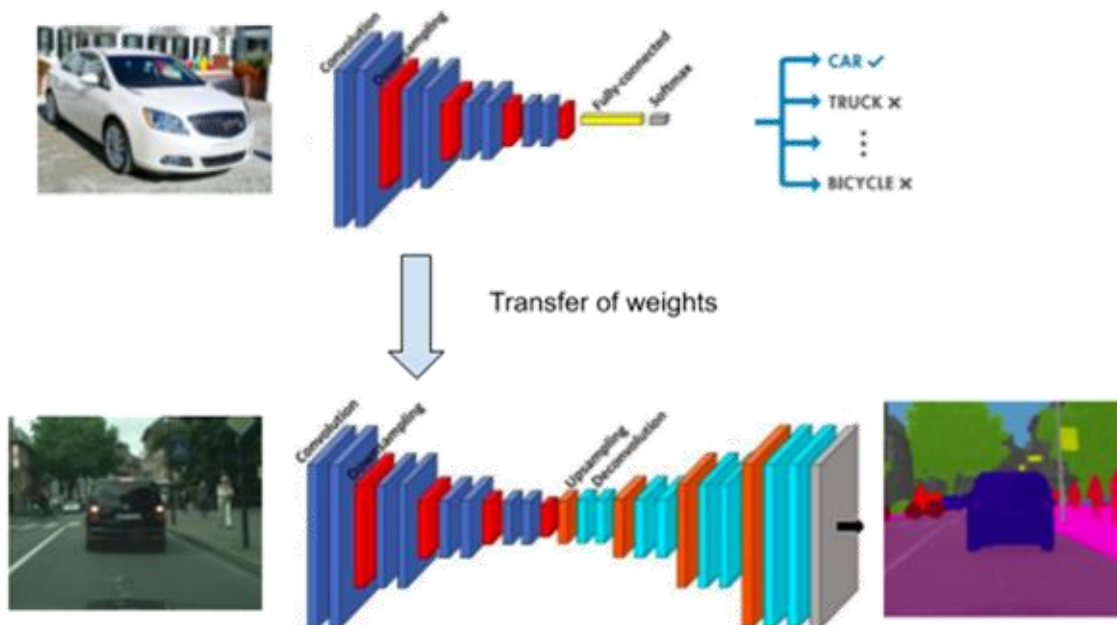
Processo de Dropout

- ▶ É uma técnica de regularização usada para reduzir o efeito de overfitting
- ▶ Aleatoriamente, com uma probabilidade p , alguns nós são ignorados
- ▶ Faz com que a rede não dependa muito de nós individuais e generalize mais o modelo de classificação



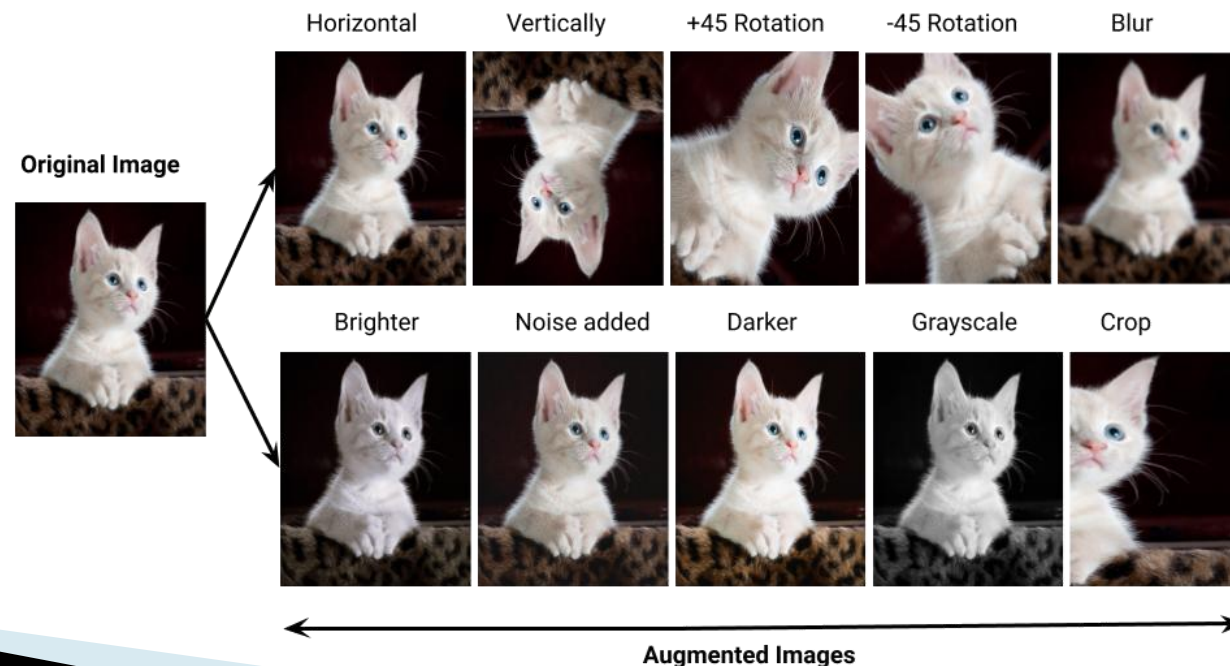
Transferência da Aprendizagem

- ▶ Treinar toda a CNN pode levar semanas e é preciso de fazer sempre que se tenta resolver um novo problema
- ▶ Uma solução é **transferir os pesos** de redes pré-treinadas (AlexNet, ResNet e outros) para gerar os mapas de características
- ▶ Depois só é necessário alterar as camadas finais para se adaptar ao seu caso particular



Data Augmentation

- ▶ Processo artificial de aumento da dimensão do conjunto inicial de imagens que consiste em criar novas imagens por meio de uma transformação geométrica, cor, intensidade, ou outra
- ▶ Objetivo é criar um classificador mais robusto e com menos overfitting



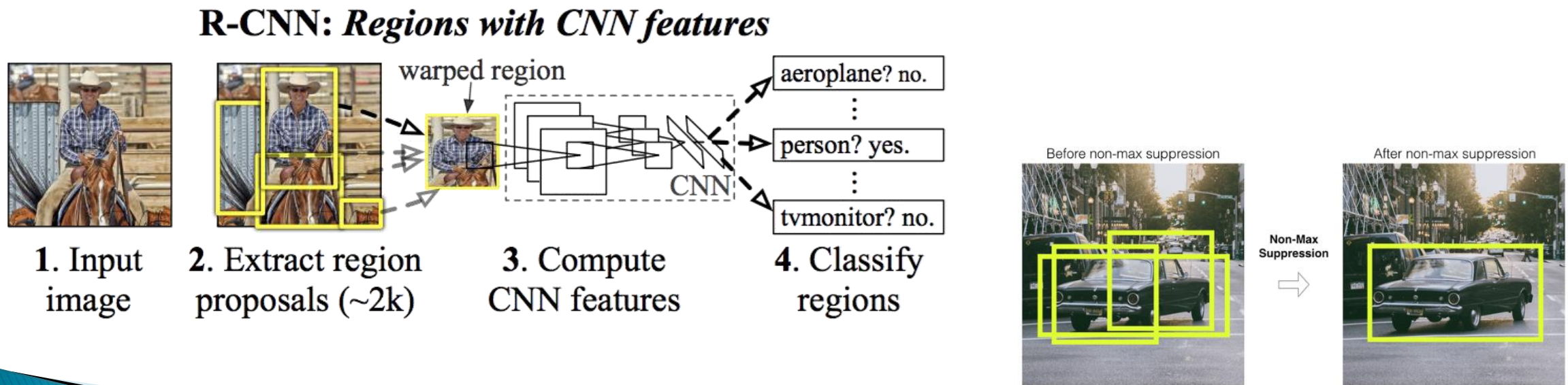
Principais Arquitecturas

17/11/2025

André D. Mora TAPDI – 2025/2026
DEEC/FCT/UNL

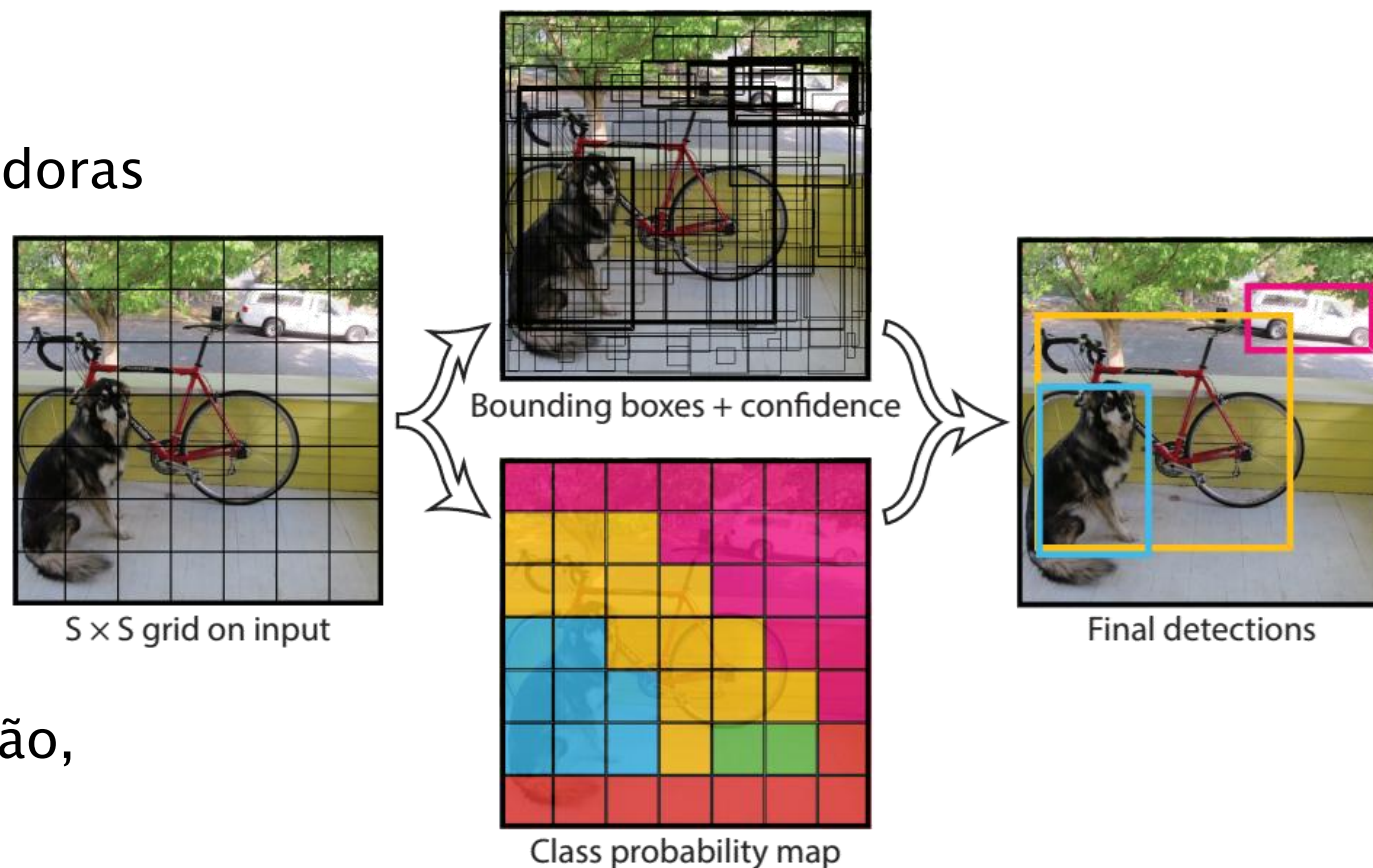
R-CNN – Region based CNN

- ▶ Publicada em 2014
 - Pioneira dos métodos: Fast-CNN, Faster-CNN, Mask-CNN
- ▶ O algoritmo inclui um passo de detecção de regiões de interesse que são depois classificadas por uma rede CNN



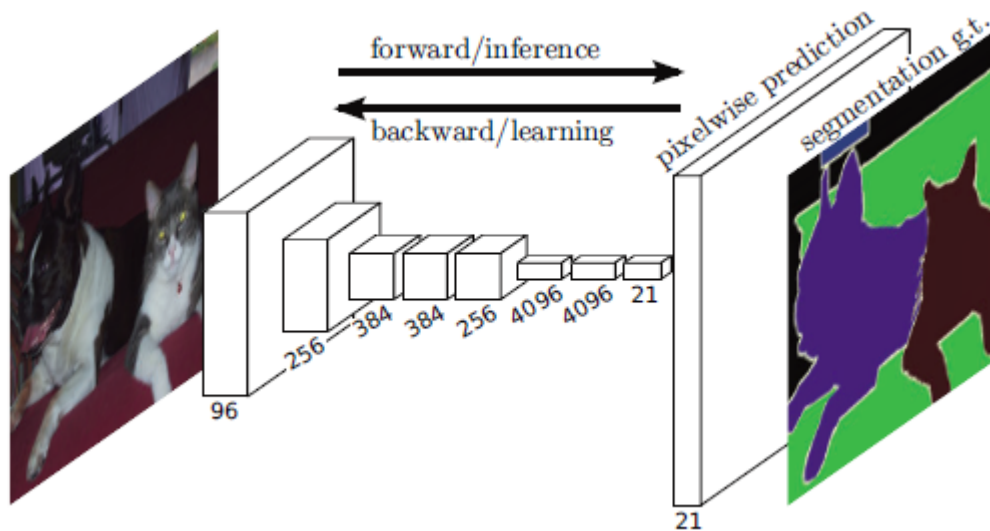
YOLO um classificador já treinado

- ▶ YOLO – You Only Look Once
 - Rápido
 - Menor precisão
 - Disponibiliza as caixas delimitadoras
- ▶ Proposto em 2015
- Versões populares:
 - V3
 - V4
 - ...
 - V12 – Feb, 2025
- ▶ Encontram-se versões do modelo mais compactas com menor precisão, mas maior velocidade (tiny, nano)



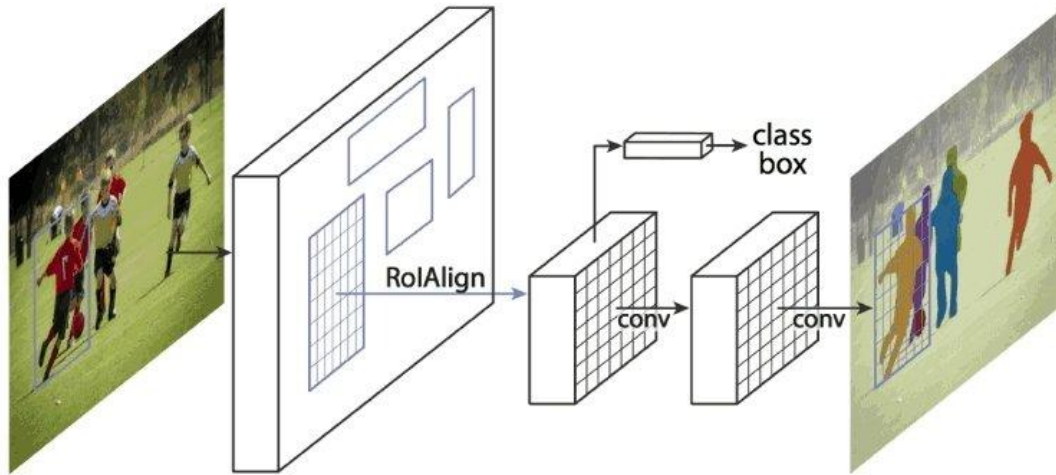
FCN – Fully Convolutional Network for Semantic Segmentation

- ▶ Arquitetura autoencoder
- ▶ Publicada em 2014



Mask R-CNN

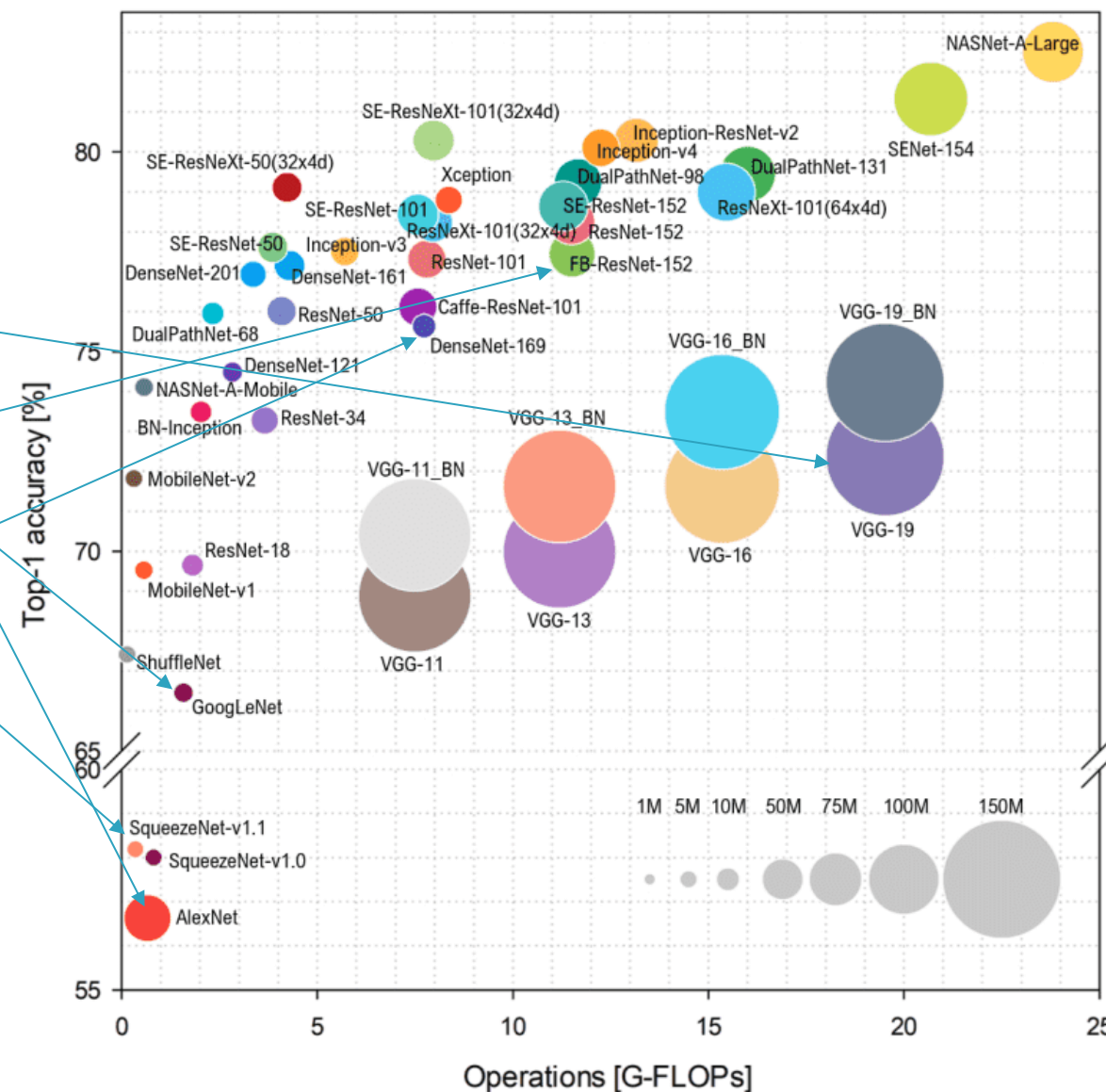
- ▶ Arquitectura autoencoder
- ▶ Permite fazer object detection and instance segmentation



Outras CNNs populares

AlexNet (8 layers) ●
VGG-19 (19 layers) ●
GoogLeNet – (22 layers) ●
ResNet ●
Squeeze Net ●
DenseNet ●

Redes pré-treinadas com imageNet podem
classificar imagens em 1000 classes



Implementação

17/11/2025

André D. Mora TAPDI – 2025/2026
DEEC/FCT/UNL

Plataformas – API

- Existem plataformas/APIs (muitas em open-source) que auxiliam a criação e o treino de modelos de CNN, algumas das mais populares são:

 TensorFlow	PYTORCH	 Keras	DEEPLEARNING4J	Caffe
Google	Meta (Facebook)	Interface fácil	Para Java	Hard to use

 ONNX
Open Neural Network Exchange (Microsoft and Facebook) Foco na interoperabilidade

PyTorch

AlexNet example

```
class AlexNet(nn.Module):
    def __init__(self, num_classes: int = 1000) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x
```

TensorFlow

- ▶ TensorFlow é uma *high-level APIs*, baseada no Keras para desenhar e treinar redes neurais
- ▶ A fácil e intuitiva API do Keras permite uma rápida prototipagem, seja para investigação ou produção

Exemplo de estrutura

```
#create model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation="relu", input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10)
])
```

TensorFlow (2)

► Processo

- Dataset creation
 - keras has 7 datasets available
 - MNIST – digit
- Network structure
 - Layers
 - Configs
- Network compilation
- Train
- Predict

```
# load data – MNIST digits dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# create model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation="relu", input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])

# compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# train
model.fit(x_train, y_train, epochs=5)

# test
model.evaluate(x_test, y_test, verbose=2)
```



TensorFlow (3) + OpenCV

- ▶ Para usar o modelo no OpenCV é necessário gravá-lo num formato reconhecido pelo OpenCV

```
from tensorflow.python.framework.convert_to_constants import convert_variables_to_constants_v2

#Save to OpenCV
tf_model_graph = tf.function(lambda x: model(x))
tf_model_graph = tf_model_graph.get_concrete_function(tf.TensorSpec(model.inputs[0].shape, model.inputs[0].dtype))

# obtain frozen concrete function
frozen_tf_func = convert_variables_to_constants_v2(tf_model_graph)
frozen_tf_func.graph.as_graph_def()

# save full TF model
tf.io.write_graph(graph_or_graph_def=frozen_tf_func.graph,
                  logdir=".\\",          name="modelo_CNN_digitos_TF.pb",          as_text=False)
```


Em OpenCV... DNN module

TensorFlow
Caffe
Torch
DarkNet
ONNX
ModelOptimizer

```
net = cv2.dnn.readNetFromTensorflow(modelFile, configFile)
```

```
# prepare for DNN
```

```
blob = cv2.dnn.blobFromImage(frameOpencvDnn, 1.0, (500, 300), [104, 117, 123], True, False)
```

```
#set image as DNN input
```

```
net.setInput(blob)
```

```
#get Output
```

```
detections = net.forward()
```

Image preprocessing for DNN

crop

Swap Red and Blue Channels

Mean value for RGB

Output Image size

Scale Factor

Depending on the network struture

Example of face detetion

First blob

- [a,b,
- detections,
- [image, class, confidence score, StartX, StartY, EndX, EndY]]s